

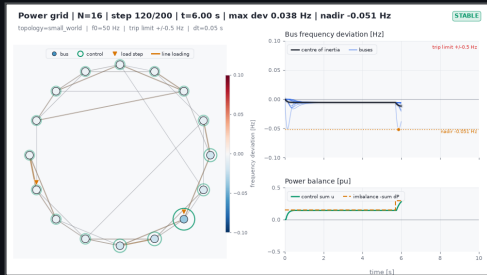
My Tool Box

Networked multi-agent control
environments, classical MARL
algorithms and research tools

Gymnasium API · continuous states and actions
native batched simulation · reference controllers
MAPPO, MADDPG, QMIX and more, trained on the
environments

Dongming Wang

Version 1.2 · github.com/EricDMW/My_Tool_Box



At a glance

8

environment families
(18 registered ids)

7

classical MARL algorithms
trained on the environments

2–3 M

agent-steps per second
(256 copies, one CPU core)

1 462

tests, CI on
Python 3.9 to 3.13

env_lib simulation

Oscillator networks, consensus and formation, power-grid frequency control, vehicle platoons, multi-robot target tracking, cooperative physics and communication benchmarks, behind one Gymnasium interface; 6 families with continuous states and actions.

marl_algorithms learning

IPPO, MAPPO, MADDPG, MATD3, IQL, VDN and QMIX on one small core, trained on batched copies of the environments and evaluated against their classical controllers.

toolkit research workflow

Publication-quality plots with confidence bands, PyTorch policy and value networks, and reproducible experiment parameters, in the same install.

Built for networked multi-agent control

Interaction structure matters

Every networked environment exposes its graph (`env.adjacency`); shared topologies and k -hop neighbourhoods in `env_lib.utils.graphs`.

Physics, not grid worlds

Continuous states and actions in physical units: swing equations, vehicle dynamics, coupled oscillators, integrators, range-bearing sensing.

Many agents, many samples

Batch-first kernels: `make_vec(id, 1024)` advances every copy with a few array operations; a PyTorch backend runs on GPUs.

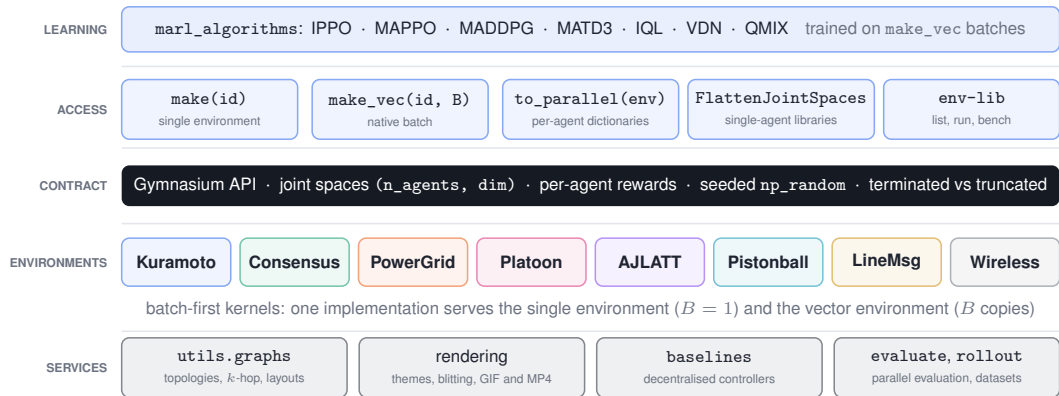
Credit and comparison

Per-agent rewards at every step, and a classical decentralised controller for every environment as the reference to beat.

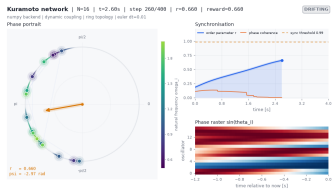
Results others can reproduce

Seeded generators only, headless rendering and GIF/MP4 export, plotting and parameter tools in the same package.

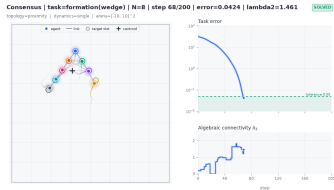
Design



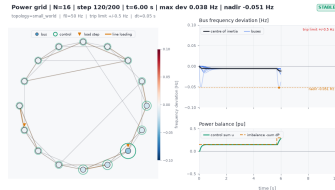
Environment suite



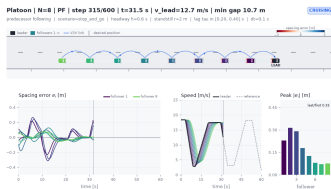
Kuramoto synchronisation



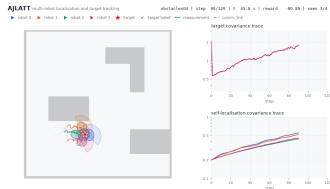
Formation control



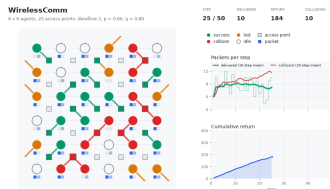
Power-grid frequency control



Vehicle platoon (CACC)



Multi-robot target tracking



Wireless access grid

Continuous states and actions

Environment	State and dynamics	Action per agent	Interaction graph
PowerGrid	bus phase angles and frequencies; swing equations	bounded power injection	transmission network
Platoon	gaps, speeds and accelerations; actuator lag	commanded acceleration	V2V topology
Consensus, Formation	planar positions (and velocities); single or double integrator	velocity or acceleration	fixed or proximity graph
Kuramoto	oscillator phases; phase-coupled dynamics	control input and coupling gains	coupling network
AJLATT	robot and target poses, EKF beliefs	linear and angular velocity	communication range
Pistonball	rigid-body physics (pymunk)	piston velocity (or discrete)	line of pistons

Observations are stacked per agent, (`n_agents`, `obs_dim`) `float32`, in physical units with documented layouts (`env.observation_layout`). `LineMsg` and `WirelessComm` add discrete networked benchmarks.

One interface, from a single run to large batches

Python

```
import env_lib

env = env_lib.make("PowerGrid-v0")
obs, info = env.reset(seed=0)      # (agents, obs_dim)
policy = env_lib.baseline_policy(env)
obs, r, term, trunc, info = env.step(policy(obs))
info["agent_rewards"]              # (agents,)
```



```
envs = env_lib.make_vec("PowerGrid-v0", num_envs=1024)
obs, _ = envs.reset(seed=0)        # (1024, agents, dim)
print(env_lib.evaluate(envs, policy, n_episodes=1024))
```

Existing learners

```
from env_lib.wrappers import (
    FlattenJointSpaces, to_parallel)
sb3_env = FlattenJointSpaces(env)
pettingzoo_env = to_parallel(env)
```

Shell

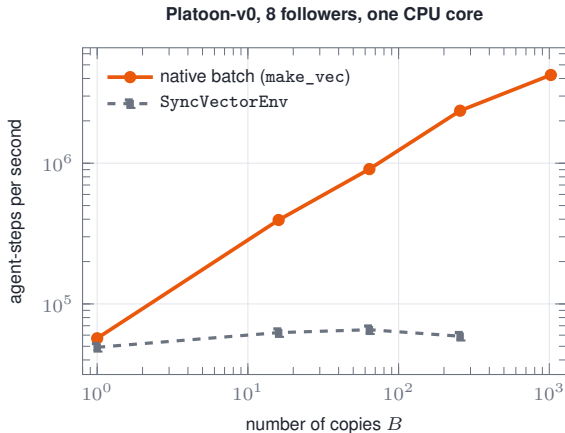
```
$ env-lib list --continuous
$ env-lib describe Platoon-v0
$ env-lib run Platoon-v0 --gif f.gif
$ env-lib evaluate PowerGrid-v0
$ env-lib bench Platoon-v0
```

Batch-first simulation

Each continuous environment is written once, for arrays with a leading batch axis:

$$x_{t+1} = f(x_t, u_t), \quad x_t \in \mathbb{R}^{B \times n \times d}.$$

- single environment: $B = 1$;
- vector environment: B copies, same array operations;
- Gymnasium autoreset modes;
- copy 0 reproduces the single environment.



Single environment, time per step

Environment	before 1.0	1.2
Kuramoto, 50 oscillators	2.36 ms	0.09 ms
Kuramoto (PyTorch), 50×8	10.3 ms	0.45 ms
WirelessComm, 12×12	0.56 ms	0.06 ms
Pistonball, 20 pistons	1.11 ms	0.08 ms
AJLATT, 4 robots	90 ms	4.7 ms
PowerGrid, 16 buses	new	0.25 ms
Platoon, 8 followers	new	0.13 ms

Where the speed comes from

- vectorised dynamics, no Python loops over agents;
- exact batched ray casting on occupancy grids;
- covariance intersection by projected Newton with analytic derivatives;
- persistent artists and blitting for rendering;
- optional dependencies imported only when used.

Seeded trajectories were checked against the previous implementation wherever the refactor was not meant to change results.

A reference controller for every environment

Environment	Decentralised baseline	Random	Baseline
PowerGrid	droop control $u_i = -k \omega_i$	-562	-0.76
Platoon	CACC $u_i = k_p e_i + k_d \Delta v_i + k_a a_{i-1}$	-5604	-46
Consensus	Laplacian protocol	-17327	-1414
Formation	Laplacian protocol on formation offsets	-18166	-1608
Kuramoto (penalty reward mode)	frequency compensation and phase feedback	3.2	6.9
AJLATT (no collision termination)	encircle the target belief	-34273	-7415
Pistonball	ramp towards the ball	-254	801
LineMsg	always relay	43	95
WirelessComm	collision-free access schedule	388	924

Mean return on the same 64 seeded episodes (`benchmark_baselines.py`; higher is better).

`env_lib.baseline_policy(env)` returns the controller as a function of the observation, so the same call drives one environment or a batch of thousands.

Classical MARL algorithms, trained on the environments

Algorithm	Family, actions
IPPO, MAPPO	on-policy; continuous or discrete
MADDPG, MATD3	off-policy actor-critic; continuous
IQL, VDN, QMIX	value-based; discrete

One core serves all seven: per-agent view of the joint spaces, batched experience collection, buffers, shared or per-agent networks, two training loops.

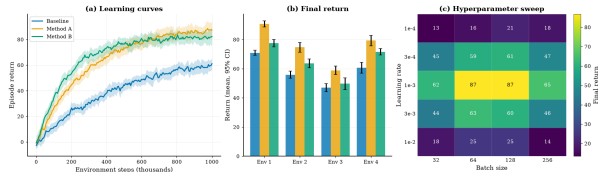
```
$ marl-train run mappo PowerGrid-v0
$ marl-train compare PowerGrid-v0 \
  --seeds 0 1 2      # baselines
```

Tuned presets, one CPU core, 64 seeded episodes

Algorithm	Environment	Time	Random	Trained	Baseline
MAPPO	PowerGrid	106 s	−629	−0.89	−0.79
MAPPO	Platoon	102 s	−5350	−65	−46
IPPO	Consensus	83 s	−18041	−1550	−1518
MADDPG	Consensus	72 s	−18041	−1681	−1518
VDN	Wireless	74 s	139	338	340
QMIX	LineMsg	16 s	43	95	95

Mean return, higher is better; the baseline is the environment's classical controller.

Research utilities in the same install



plotkit learning curves with confidence bands, grouped bars, heatmaps, colour-blind-safe palette, vector export.

neural_toolkit MLP, CNN, RNN and Transformer policies, value and Q-networks, encoders and decoders (PyTorch).

parakit save, load and validate argparse experiment parameters; optional editor.

recording `record_episode(env, policy, "run.gif")` for every environment, in dark and light themes.

Engineering quality

PACKAGING

One `pyproject.toml` and `src` layout; heavy dependencies as extras (`torch`, `pistonball`, `video`); lazy imports keep `import env_lib` light; console scripts.

REPRODUCIBILITY

Randomness only from seeded generators, never global state; headless rendering; rollouts saved as `.npz`; parameters saved and reloaded with `parakit`.

CORRECTNESS

1 462 tests: Gymnasium API contract, seeded determinism, batch versus single equivalence, physics sanity checks; continuous integration on Python 3.9 to 3.13.

DOCUMENTATION

Handbook of 229 pages with model equations, observation layouts and parameter tables; API reference and migration guide; an example script per environment.

Get started

```
$ git clone https://github.com/EricDMW/My_Tool_Box
$ cd My_Tool_Box && pip install -e ".[all]"
$ env-lib list
$ env-lib run Platoon-v0 --gif platoon.gif
```

Slides	docs/slides/my_tool_box_slides.pdf
Handbook	docs/manual/main.pdf
Examples	examples/ (grouped by topic)
Changes	CHANGELOG.md

